



UNIVERSIDADE DA CORUÑA

Facultade de Informática

Máster Universitario en Enxeñaría en Informática

TRABALLO FIN DE MÁSTER

Aplicación para la Gestión de Emergencias

Estudiante: Adrián García Vázquez

Dirección: Manuel Álvarez Díaz

Monforte de Lemos, abril de 2026.

Dedicatoria

Agradecimientos

Gracias a Manuel, mi tutor, por su paciencia infinita y sus consejos que me han permitido hacer realidad este proyecto.

Gracias a mi familia, mis amigos y a mi novia por el apoyo incondicional y ánimos.

También agradezco a mis compañeros y profesorado de MUEI que me han ayudado a mejorar como informático y como persona.

Resumen

Galicia afronta de forma recurrente emergencias de alto impacto (incendios, inundaciones, derrumbes y riesgos industriales), que exigen una respuesta rápida y coordinada. En este contexto, la gestión operativa se ve limitada por la fragmentación de herramientas, la escasa integración de datos y la necesidad de coordinar a múltiples actores en escenarios de alta presión.

Este Trabajo de Fin de Máster parte de un sistema previo centrado en incendios forestales y lo amplía hacia una plataforma de coordinación multi-riesgo para Galicia. La solución se apoya en dos pilares: una plataforma web evolucionada y una aplicación móvil nativa en Kotlin para uso en campo y participación ciudadana. El sistema centraliza alertas y emergencias, integra geolocalización y reportes multimedia, coordina recursos y ofrece visualización cartográfica en tiempo real de eventos y despliegues. Además, incorpora notificaciones push y un histórico para análisis post-emergencia, con indicadores de tiempos de respuesta y recursos utilizados. Con ello, el proyecto mejora la interoperabilidad, optimiza la toma de decisiones y refuerza la capacidad de respuesta en el territorio gallego.

Abstract

Galicia recurrently faces high-impact emergencies (fires, floods, landslides, and industrial hazards) that require fast, coordinated response. In this context, operational management is limited by fragmented tools, weak data integration, and the need to coordinate multiple stakeholders in high-pressure scenarios.

This Master's Thesis extends a previous wildfire-focused system into a multi-risk coordination platform for Galicia. The solution is built on two pillars: an enhanced web platform and a native Kotlin mobile app for field use and citizen participation. The system centralizes alerts and incidents, integrates geolocation and multimedia reporting, supports resource coordination, and provides real-time map visualization of events and deployments. It also includes push notifications and historical records for post-incident analysis, with indicators on response times and resource usage. Overall, the project improves interoperability, supports better decision-making, and strengthens emergency response capacity across the Galician territory.

Palabras clave:

- Galicia
- Gestión de emergencias
- Geolocalización
- Coordinación de personas y recursos
- Aplicación Web
- Java - Spring Boot
- React - Redux
- Aplicación móvil Kotlin

Keywords:

- Galicia
- Emergency management
- Geolocation
- Aplicación Web
- Coordination of people and resources
- Java
- React - Redux
- Kotlin mobile application

Índice general

1	Introducción	1
1.1	Determinación de la situación actual	1
1.2	Alcance y objetivos	2
1.2.1	Objetivos principales	3
1.2.2	Objetivos concretos	3
1.3	Estructura de la memoria	4
2	Base Tecnológica	5
2.1	Lenguajes	5
2.1.1	Java	5
2.1.2	Kotlin	5
2.1.3	SQL	5
2.1.4	HTML Y CSS	5
2.1.5	JavaScript	6
2.1.6	JSX	6
2.1.7	LaTeX	6
2.1.8	JSON	6
2.2	Frameworks y librerías servidor	7
2.2.1	Spring	7
2.2.2	SpringBoot	7
2.2.3	SpringData	7
2.2.4	JPA	7
2.2.5	Hibernate	7
2.2.6	JDBC	7
2.3	Frameworks y librerías web	8
2.3.1	React	8
2.3.2	Redux	8

2.3.3	RTKQuery	8
2.3.4	MaterialUI	8
2.3.5	React Map Gl	8
2.3.6	Open Weather Map	8
2.3.7	Mapbox	9
2.4	Frameworks y librerías web	9
2.4.1	Gradle	9
2.4.2	JetPack Compose	9
2.4.3	Material Design	9
2.4.4	HttpClient	9
2.4.5	Firebase Cloud Messaging	10
2.4.6	FirebaseMessagingService	10
2.5	Pruebas	10
2.5.1	JUnit	10
2.6	Protocolos	10
2.6.1	HTTP/S	10
2.6.2	REST	10
2.7	Herramientas de desarrollo	11
2.7.1	IntelliJ	11
2.7.2	Postman	11
2.7.3	VSCode	11
2.7.4	DBeaver	11
2.7.5	GIT	11
2.7.6	Apache Maven	12
2.7.7	Create React App	12
2.8	Sistemas de gestión de base de datos	12
2.8.1	PostgreSQL	12
2.8.2	PostGIS	12
2.9	Tecnologías de alojamiento de servicios	12
2.9.1	Tomcat	12
2.9.2	Webpack	12
2.9.3	Serve	13
2.9.4	Docker	13
3	Estado del arte	14
4	Análisis de viabilidad	15

5	Introducción al desarrollo realizado	16
5.1	Introducción	16
5.2	Arquitectura de la Solución	16
5.3	Metodología e iteraciones	16
6	Viabilidad y planificación	17
6.1	Análisis de viabilidad	17
6.2	Planificación	17
6.3	Seguimiento	17
6.4	Análisis de costes	18
7	Requisitos del sistema	19
7.1	Introducción	19
7.2	Actores	19
7.3	Casos de uso	19
7.4	Modelo de casos de uso	19
7.5	Prototipado de interfaz	20
8	Diseño	21
8.1	Introducción y objetivos	21
8.2	Resumen de patrones usados	21
8.3	Arquitectura general	21
8.4	Subsistema Backend	21
8.4.1	Objetivos	21
8.4.2	Arquitectura	21
8.4.3	Modelo del dominio	22
8.4.4	Capa de acceso a datos	22
8.4.5	Capa de lógica de negocio	23
8.4.6	Capa servicios	23
8.4.7	Otros?	23
8.5	Subsistema Frontend	23
8.5.1	Objetivos	23
8.5.2	Arquitectura	23
8.5.3	Capa de acceso al servicio	24
8.5.4	Capa de componentes de interfaz	24
8.5.5	Otros?	24

9 Implementación	25
9.1 Software requerido	25
9.2 Estructura	25
9.3 Instrucciones de compilación	25
10 Pruebas	26
10.1 Introducción	26
10.2 Pruebas unitarias	26
10.3 Pruebas de integración	26
10.4	27
11 Conclusiones y futuras líneas de trabajo	28
11.1 Conclusiones	28
11.2 Aprendizajes	28
11.3 Futuras líneas de trabajo	28
A Material adicional	30
A.1 Instalación del Software	30
A.2 Manual de usuario	30
Lista de acrónimos	31
Bibliografía	33

Índice de figuras

Índice de tablas

Introducción

GALICIA presenta una elevada exposición a emergencias en su territorio, entre las que destacan especialmente los incendios forestales, las inundaciones derivadas de episodios de lluvia intensa, los derrumbes de carreteras o terraplenes y determinados riesgos industriales. En los últimos años han surgido emergencias que ilustran su importancia y su impacto operativo: campañas con incendios que han afectado a superficies significativas según la estadística oficial [1]; episodios de abril de 2026 que motivaron una decena de incidencias en Ourense y la activación de alertas en las Rías Baixas [2, 3]; y movimientos sísmicos locales perceptibles por la población, como el registrado en Ponteceso en 2026 [4]. La complejidad operativa de la respuesta exige coordinación entre múltiples actores, toma de decisiones en ventanas temporales reducidas y disponibilidad de información operativa fiable. Estos retos son tanto tecnológicos como organizativos y normativos, ya que confluyen distintos niveles administrativos y tecnologías

Este [Trabajo Fin de Máster \(TFM\)](#) se apoya y amplía el trabajo previo desarrollado en el [Trabajo Fin de Grado \(TFG\)](#) "Aplicación para coordinación de equipos para la prevención y extinción de incendios forestales" [5], que documenta el diseño e implementación de una aplicación web destinada a centralizar avisos, recursos y despliegues en el contexto de incendios forestales en Galicia. El [TFG](#) aporta decisiones y soluciones de diseño que se toman como punto de partida en este [TFM](#).

1.1 Determinación de la situación actual

La situación actual combina capacidades consolidadas de detección, alerta y despliegue con limitaciones de integración entre sistemas y organizaciones. En el ámbito forestal, existen estadísticas nacionales y autonómicas que permiten caracterizar la magnitud del problema [1]. Galicia dispone además de una estructura operativa asentada en torno a [Axencia Galega de Emerxencias \(112 Galicia\)](#) ([AXEGA](#)) y cuenta con planes de emergencia por tipología de riesgo

a nivel autonómico [6, 7].

Galicia dispone de una organización y planificación detallada para distintas tipologías de riesgo —por ejemplo, [Plan de Prevención y Defensa Contra los Incendios Forestales de Galicia \(PLADIGA\)](#) para riesgos forestales—. La Consellería de Presidencia centraliza catálogos, protocolos y guías municipales; la Consellería de Medio Rural publica documentación técnica y memorias sobre defensa del monte; y [AXEGA](#) mantiene boletines e instrumentos operativos. Sin embargo, estos planes y protocolos están dispersos entre distintos organismos y repositorios, y no están unificados en una solución digital compartida y accesible tanto para los equipos de gestión de emergencias como para la ciudadanía común [6, 7, 8].

No obstante, persisten dificultades operativas que limitan la eficacia conjunta:

- **Fragmentación de la información:** avisos ciudadanos, información meteorológicas, mapas de emergencias y estado de recursos residen en sistemas segregados.
- **Heterogeneidad organizativa:** organismos y equipos tienen procedimientos y necesidades distintas (coordinación estratégica, mando táctico, intervención en campo).
- **Variabilidad del riesgo y concurrencia de eventos:** no es realista el tratamiento aislado de las emergencias, ciertas emergencias suelen estar relacionadas con otras emergencias, exigiendo un modelo multi-riesgo [3, 7].
- **Presión temporal y trazabilidad:** la necesidad de decisiones rápidas hace imprescindible la actualización en tiempo real y el registro de actuaciones para análisis posteriores.

El análisis realizado en el [TFG](#) muestra que estas limitaciones no son debidas a la ausencia de medios, sino a falta de cohesión operativa digital. Por ello, este [TFM](#) propone una evolución del sistema desarrollado en el [TFG](#) hacia una plataforma de coordinación que mejore la integración, la trazabilidad y el soporte a la decisión en contextos multi-riesgo.

1.2 Alcance y objetivos

El alcance de este [TFM](#) es evolucionar la solución técnica y los modelos operativos desarrollados en el [TFG](#) hacia una plataforma de coordinación de emergencias multi-riesgo para Galicia, con dos componentes principales: una plataforma web para gestión y coordinación, y una aplicación móvil nativa para uso en campo y participación ciudadana. Este [TFM](#) parte explícitamente del trabajo técnico y de campo documentado en el [TFG](#) [5].

La solución propuesta se alinea con el trabajo realizado por protección civil y otros organismos de respuesta a emergencias, priorizando la interoperabilidad entre estos organismos, sin pretender sustituir sistemas institucionales existentes, sino complementarlos aportando una capa unificada de gestión, visualización y soporte a la decisión [9, 10].

1.2.1 Objetivos principales

A continuación se enumeran los principales objetivos que debe cumplir la aplicación a desarrollar en este trabajo:

1. Extender el dominio funcional del TFG hacia un enfoque multi-riesgo (incendios, inundaciones, derrumbes y riesgos industriales) acorde con los planes autonómicos.
2. Ampliar funcionalidades de la plataforma web existente que centraliza la gestión de alertas, emergencias, recursos y despliegues operativos.
3. Diseñar e implementar una aplicación móvil nativa en Kotlin para reporte ciudadano y soporte a la intervención en campo.
4. Mejorar la coordinación multi-organizativa, integrando en un mismo flujo de trabajo a coordinadores, brigadas, protección civil y ciudadanía.
5. Reforzar la trazabilidad y el análisis post-emergencia mediante históricos e indicadores operativos.

1.2.2 Objetivos concretos

1. Contextualización: esclarecer el valor aportado por el TFM y definir el nuevo dominio multi-riesgo.
2. Definir la arquitectura de la aplicación móvil y entregar un esqueleto que permita añadir funcionalidades progresivamente.
3. Introducir un modelo unificado de recursos y organizaciones y definir sus capacidades y área operativa.
4. Permitir la creación y edición de emergencias (por punto o por cuadrante) y la creación o propuesta manual de asignaciones.
5. Introducir un registro de logs operativos de las emergencias y de las asignaciones de recursos que permita el análisis posterior de las emergencias y la generación de indicadores.
6. Integrar notificaciones móviles mediante [Firebase Cloud Messaging \(FCM\)](#), registrar dispositivos y definir el flujo de notificación y aceptación de asignaciones.
7. Implementar el almacenamiento de protocolos (por ejemplo, campo ()) y un motor de reglas capaz de generar propuestas de asignación automáticamente.

1.3 Estructura de la memoria

La memoria se organiza en once capítulos y anexos. A continuación se ofrece un resumen del contenido de cada capítulo:

1. Capítulo 1: Presenta el contexto general, describe la situación actual en Galicia, expone las motivaciones del trabajo y enuncia los objetivos generales y concretos del proyecto.
2. Capítulo 2 (2): Detalla la base tecnológica empleada: componentes de backend y frontend, bases de datos, servicios y librerías relevantes, y las decisiones arquitectónicas que sustentan la implementación.
3. Capítulo 3 (3): Revisa el estado del arte, compara soluciones existentes y extrae lecciones aplicables al diseño de la plataforma propuesta.
4. Capítulo 4 (4): Analiza la viabilidad del proyecto desde las perspectivas técnica, organizativa y normativa, e incluye una estimación de recursos y riesgos.
5. Capítulo 5 (5): Documenta el desarrollo realizado, describiendo la arquitectura global del sistema, los módulos principales y las integraciones implementadas.
6. Capítulo 6 (6): Presenta la planificación temporal del trabajo, los hitos y la metodología seguida para organizar las iteraciones de desarrollo.
7. Capítulo 7 (7): Especifica los requisitos funcionales y no funcionales que guían el diseño y la implementación de la plataforma.
8. Capítulo 8 (8): Describe el diseño funcional y técnico, incluyendo el modelo de datos, las APIs, los flujos de interacción y las decisiones de UX/UI.
9. Capítulo 9 (9): Detalla la implementación: estructura de código, despliegue, configuraciones y ejemplos de componentes desarrollados.
10. Capítulo 10 (10): Documenta las pruebas realizadas (unitarias, de integración y de aceptación), su cobertura y los resultados obtenidos.
11. Capítulo 11 (11): Resume las conclusiones, las aportaciones del trabajo y propone líneas futuras de mejora y continuidad.

Capítulo 2

Base Tecnológica

A continuación se muestra una posible agrupación de las tecnologías usadas junto con una breve descripción de cada una de ellas.

2.1 Lenguajes

2.1.1 Java

Java [11] es un lenguaje de programación orientado a objetos, que se caracteriza por su portabilidad, seguridad y facilidad de uso. Se utiliza ampliamente en el desarrollo de aplicaciones empresariales, juegos, aplicaciones móviles y aplicaciones web, entre otros.

2.1.2 Kotlin

Kotlin [12] es un lenguaje de programación moderno y conciso que se ejecuta en la máquina virtual de Java. Se ha utilizado en este proyecto para la aplicación Android debido a su interoperabilidad con Java y su sintaxis más limpia y expresiva.

2.1.3 SQL

Structured Query Language (SQL)[13] es un lenguaje de consulta estructurado utilizado para administrar y manipular bases de datos relacionales. Se utiliza para realizar tareas como la inserción, actualización y eliminación de datos, así como para la generación de informes y análisis de datos. Es uno de los lenguajes más utilizados en el mundo de la informática y es esencial para la administración y gestión de datos.

2.1.4 HTML Y CSS

HyperText Markup Language (HTML) y Cascading Style Sheet (CSS) [14] son lenguajes utilizados en la creación de páginas web. HTML se utiliza para estructurar el contenido de la

página, mientras que [CSS](#) se utiliza para darle estilo y presentación. [HTML](#) utiliza una serie de etiquetas para definir la estructura de la página, como títulos, encabezados, párrafos y enlaces, entre otros. [CSS](#) se utiliza para dar formato y estilo a la página como colores, fuentes, márgenes y bordes.

2.1.5 JavaScript

JavaScript [15] es un lenguaje de programación dinámico y no fuertemente tipado, utilizado principalmente en el desarrollo de aplicaciones web interactivas y dinámicas. Se utiliza para agregar funcionalidad a las páginas web, como animaciones, validación de formularios y efectos visuales.

2.1.6 JSX

JSX [16] es una extensión de sintaxis utilizada en la biblioteca de JavaScript React. JSX permite a los desarrolladores escribir código JavaScript que se parece mucho al marcado HTML. Esta sintaxis especial facilita la creación de componentes de interfaz de usuario reutilizables en React. Cuando se compila, JSX se traduce a llamadas regulares de JavaScript que manipulan elementos virtuales.

2.1.7 LaTeX

LaTeX [17] es un sistema de composición de documentos ampliamente utilizado en el ámbito académico y científico. Con LaTeX, los usuarios pueden crear documentos de alta calidad con una estructura lógica y una apariencia profesional.

2.1.8 JSON

[JavaScript Object Notation \(JSON\)](#) [18] es un formato de intercambio de datos textual, ligero y ampliamente utilizado. Se basa en una estructura de pares clave-valor y se utiliza para representar información estructurada de manera legible tanto para humanos como para máquinas. JSON es especialmente popular en el desarrollo de aplicaciones web, donde se utiliza para transmitir datos. Su sintaxis simple y su facilidad de uso lo convierten en una opción común para el intercambio de datos en muchas aplicaciones modernas.

2.2 Frameworks y librerías servidor

2.2.1 Spring

Spring [19] es un framework de desarrollo de aplicaciones de código abierto para Java. Proporciona una amplia gama de características y funcionalidades que facilitan el desarrollo de aplicaciones empresariales robustas y escalables. Spring se basa en el patrón de diseño de [Inversión de Control \(IoC\)](#) y el contenedor de [IoC](#) de Spring gestiona la creación y administración de objetos en una aplicación. Además del contenedor de [IoC](#), Spring ofrece módulos para la integración con bases de datos, servicios web, seguridad, transacciones y más.

2.2.2 SpringBoot

Spring Boot [20] es un framework para desarrollar aplicaciones en Java con Spring. Proporciona una configuración predeterminada que puede ser personalizada y está diseñado para simplificar el desarrollo de aplicaciones Spring al eliminar gran parte de la complejidad relacionada con la configuración.

2.2.3 SpringData

Spring Data [21] es un framework que simplifica el acceso a datos desde aplicaciones Java con bases de datos relacionales y no relacionales. Spring Data reduce la cantidad de código necesario para implementar el acceso a datos y proporciona una abstracción para trabajar con diferentes tipos de bases de datos.

2.2.4 JPA

[Java Persistence API \(JPA\)](#) [22] proporciona una capa de abstracción para trabajar con bases de datos relacionales en Java, permitiendo a los desarrolladores interactuar con las bases de datos utilizando objetos en lugar de [SQL](#).

2.2.5 Hibernate

Hibernate [23] proporciona una solución de mapeo objeto-relacional para Java que permite a los desarrolladores interactuar con las bases de datos utilizando objetos en lugar de [SQL](#). Hibernate implementa la especificación [JPA](#).

2.2.6 JDBC

[Java Database Connectivity \(JDBC\)](#) [24] es una [API](#) de Java que proporciona un conjunto de clases e interfaces para interactuar con bases de datos relacionales. JDBC facilita la conexión,

consulta y manipulación de datos en bases de datos a través del lenguaje de programación Java.

2.3 Frameworks y librerías web

2.3.1 React

React [25] es una biblioteca de JavaScript para construir interfaces de usuario. Proporciona una forma declarativa para construir componentes de interfaz de usuario reutilizables y altamente modulares.

2.3.2 Redux

Redux [26] es una biblioteca de JavaScript para manejar el estado de las aplicaciones. Proporciona una forma de manejar el estado de las aplicaciones, haciendo que la gestión del estado sea más fácil de entender y depurar.

2.3.3 RTKQuery

RTKQuery [27] es una biblioteca de JavaScript para manejar las solicitudes a la [API](#) en aplicaciones Redux. Proporciona una solución escalable y fácil de usar, simplificando la gestión del estado de la aplicación en respuesta a las solicitudes de la [API](#).

2.3.4 MaterialUI

Material [28] es un framework de diseño de componentes de React basado en los principios de diseño de Material Design. Proporciona una amplia variedad de componentes preconstruidos y personalizables, lo que facilita la creación de interfaces de usuario vistosas y funcionales.

2.3.5 React Map Gl

React Map GL [29] es una librería de JavaScript que se utiliza para crear mapas interactivos en aplicaciones web utilizando la biblioteca React. Esta librería se basa en Mapbox GL, una librería de mapas de código abierto que utiliza WebGL para renderizar mapas altamente interactivos y eficientes en términos de rendimiento.

2.3.6 Open Weather Map

Open Weather Map [30] es un servicio en línea que proporciona datos meteorológicos y de pronóstico en tiempo real. Permite a los desarrolladores acceder a una amplia gama

de información climática, como temperatura, humedad, velocidad del viento, condiciones atmosféricas, pronósticos a corto y largo plazo, entre otros.

2.3.7 Mapbox

Mapbox [31] es un servicio en línea que ofrece una plataforma de mapas interactivos y herramientas de geolocalización. Proporciona una amplia gama de funcionalidades, incluyendo la visualización de mapas en 2D y 3D, la capacidad de agregar marcadores y rutas, la personalización de estilos y la integración con datos geoespaciales.

2.4 Frameworks y librerías web

2.4.1 Gradle

Gradle [32] es un sistema de automatización de compilación de código abierto que se utiliza para construir, probar y desplegar aplicaciones. Gradle es altamente flexible y se puede utilizar para proyectos de cualquier tamaño, desde pequeñas aplicaciones hasta grandes sistemas empresariales.

2.4.2 JetPack Compose

Jetpack Compose (Compose) [33] permite a los desarrolladores crear interfaces de usuario de forma declarativa utilizando Kotlin. Proporciona una forma más sencilla y eficiente de construir interfaces de usuario en Android, eliminando la necesidad de escribir código XML para diseñar las pantallas.

2.4.3 Material Design

Material Design (Material) [34] es un sistema de diseño desarrollado por Google que proporciona un conjunto de principios y directrices para crear interfaces de usuario atractivas, intuitivas y consistentes en diferentes plataformas y dispositivos.

2.4.4 HttpClient

HttpClient [35] es una biblioteca de Kotlin que proporciona una forma sencilla y eficiente de realizar solicitudes HTTP. Permite a los desarrolladores enviar solicitudes HTTP, manejar respuestas y gestionar conexiones de red de manera fácil y flexible.

2.4.5 Firebase Cloud Messaging

[Firebase Cloud Messaging \(FCM\)](#) [36] es un servicio de mensajería en la nube proporcionado por Google que permite a los desarrolladores enviar notificaciones y mensajes a dispositivos móviles y aplicaciones web.

2.4.6 FirebaseMessagingService

[FirebaseMessagingService \(FMS\)](#) [37] es una clase de servicio en Android que se utiliza para manejar la recepción de mensajes y notificaciones enviados a través de [FCM](#).

2.5 Pruebas

2.5.1 JUnit

JUnit [38] es un framework de pruebas unitarias para el lenguaje de programación Java. Proporciona un conjunto de anotaciones y aserciones para facilitar la creación y ejecución de pruebas unitarias en código Java. JUnit se ha convertido en una de las herramientas más populares para la prueba de software en Java y ha sido ampliamente adoptado por la comunidad de desarrollo de software.

2.6 Protocolos

2.6.1 HTTP/S

[Hypertext Transfer Protocol \(HTTP\)](#) [39] es un protocolo de comunicación utilizado en la World Wide Web para transferir información entre servidores y clientes. Se utiliza para solicitar y enviar recursos, como páginas web, imágenes y vídeos, a través de la web. [Hypertext Transfer Protocol Secure \(HTTPS\)](#) es una extensión de [HTTP](#), que usa la encriptación para realizar las comunicaciones de manera segura.

2.6.2 REST

[Representational State Transfer \(REST\)](#) [40] es un estilo de arquitectura software para transferir información entre aplicaciones. REST se ha vuelto ampliamente adoptado en el desarrollo de servicios web debido a su simplicidad, flexibilidad y capacidad para aprovechar las capacidades existentes de [HTTP](#), orientado a la comunicación de recursos y sus representaciones.

2.7 Herramientas de desarrollo

2.7.1 IntelliJ

IntelliJ [41] es un [Integrated Development Environment \(IDE\)](#) desarrollado por JetBrains para programar en Java y otros lenguajes de programación. Proporciona herramientas para facilitar la escritura de código, la depuración y la gestión de proyectos. Además, admite una amplia variedad de complementos y herramientas para la integración con otras tecnologías.

2.7.2 Postman

Postman [42] es una herramienta de colaboración y prueba de [API](#) que permite a los desarrolladores probar, documentar y compartir sus [API](#). Permite enviar solicitudes [HTTP](#), validar respuestas y automatizar tareas relacionadas con la prueba de [API](#). Además, proporciona una interfaz fácil de usar que facilita la creación y gestión de solicitudes.

2.7.3 VSCode

VSCode [43] es un editor de código fuente desarrollado por Microsoft. Ofrece un conjunto de funciones y herramientas para facilitar la programación, como la autocompletación de código, la depuración, el control de versiones, entre otros. VSCode es muy popular entre los desarrolladores debido a su interfaz de usuario intuitiva, su extensibilidad y su amplia gama de características.

2.7.4 DBeaver

DBeaver [44] es una herramienta de gestión de bases de datos multiplataforma que admite una variedad de sistemas de bases de datos como MySQL, PostgreSQL, Oracle, SQL Server y más. Proporciona una interfaz gráfica de usuario intuitiva para realizar operaciones en la base de datos, como consultas, edición de tablas y gestión de conexiones.

2.7.5 GIT

Git [45] es un sistema de control de versiones distribuido, utilizado para registrar cambios en el código fuente durante el proceso de desarrollo del software. Permite a los desarrolladores trabajar de manera colaborativa en un proyecto y realizar un seguimiento de los cambios realizados en el código fuente. Además, proporciona características como control de versiones, control de ramas y restauración de versiones anteriores.

2.7.6 Apache Maven

Apache Maven [46] es una herramienta de gestión de proyectos ampliamente utilizada en el desarrollo de software en Java. Proporciona un enfoque declarativo y basado en configuración para la construcción, gestión de dependencias y documentación de proyectos.

2.7.7 Create React App

Create React App [47] es una herramienta de línea de comandos de Facebook que permite a los desarrolladores crear rápidamente aplicaciones web de React con una configuración predeterminada y optimizada para el desarrollo. Proporciona un entorno de desarrollo rápido y fácil de usar que incluye herramientas como Webpack y Babel para compilar y optimizar el código de la aplicación.

2.8 Sistemas de gestión de base de datos

2.8.1 PostgreSQL

PostgreSQL [48] es un sistema de gestión de bases de datos relacionales de código abierto y muy potente. Ofrece una amplia variedad de características, como soporte para funciones almacenadas, índices avanzados, transacciones y más.

2.8.2 PostGIS

PostGIS [49] es una extensión de PostgreSQL que permite el manejo de datos geoespaciales, como puntos, líneas y polígonos, y ofrece una serie de funciones para el análisis y manipulación de estos datos.

2.9 Tecnologías de alojamiento de servicios

2.9.1 Tomcat

Tomcat [50] es un servidor web y contenedor de servlets de código abierto que implementa las especificaciones de Java Servlet, [JavaServer Pages \(JSP\)](#) y [Java Expression Language \(EL\)](#). Es ampliamente utilizado para ejecutar aplicaciones web desarrolladas en Java y proporciona un entorno de ejecución seguro y escalable para estas aplicaciones.

2.9.2 Webpack

Webpack [51] es una herramienta de empaquetado de módulos de código abierto ampliamente utilizada en el desarrollo web. Su principal objetivo es tomar módulos JavaScript, junto con

sus dependencias y otros recursos, y generar paquetes optimizados y eficientes para su uso en el navegador. En concreto en este proyecto se usa Webpack como despliegue en contexto de desarrollo.

2.9.3 Serve

Serve [52] es un servidor web válido para producción, que permite servir contenido estático (páginas [HTML](#), código javascript, [CSS](#) o imágenes) de forma eficiente.

2.9.4 Docker

Docker [53] es una plataforma de contenedorización que permite empaquetar aplicaciones y sus dependencias en contenedores ligeros y portables. En este proyecto la base de datos se despliega mediante contenedores Docker (imagen oficial de PostgreSQL/PostGIS), lo que facilita la gestión del entorno de desarrollo, la reproducción de la configuración y el despliegue en entornos de integración continua.

Estado del arte

X^{xxx,...} Este capítulo es interesante incluso para proyectos que no sean puramente de investigación, como es el caso de TFG. La idea es comentar en análisis de otras aplicaciones/sistemas/herramientas que pueden ser utilizados para abordar la misma problemática, con el fin de obtener feedback e identificar posibles mejoras que permitan ayudar a justificar la aportación del sistema implementado en este TFG; aunque realmente no es necesario justificar aportación porque se trata de un trabajo clásico de ingeniería, siempre resulta positivo.

Análisis de viabilidad

X^{xxx,...}
Aspectos a considerar

- Capítulo opcional pero recomendable.
- Su objetivo es realizar un análisis de la viabilidad del proyecto, validando sus posibles riesgos.
- Suele enfocarse descomponiéndolo en diferentes dimensiones: temporal (duración estimada para un TFG), tecnológica (relacionado con tecnologías usadas, normalmente opensource), económica (relacionado con tiempo de alumno en TFG, recursos requeridos que normalmente es HW ya disponible, el SW a realizar y otro SW normalmente opensource), ...

Introducción al desarrollo realizado

EN este capítulo se resume ...
XX

5.1 Introducción

Aspectos a considerar

- Mini-resumen de lo comentado hasta ahora (1 o 2 párrafos)
- Aplicación SPA ...

5.2 Arquitectura de la Solución

Aspectos a considerar

- Diagrama de despliegue de la aplicación mostrando todas las tecnologías utilizadas en su sitio
- Justificación de tecnologías

5.3 Metodología e iteraciones

Aspectos a considerar

- Comentar en 1-3 párrafos la metodología a utilizar, identificando sus principales características - un párrafo, poco más, lo importante es comentar la metodología aplicada al proyecto NO TEORÍA GENERAL sobre la metodología.
- Finalmente se indicarán las iteraciones principales identificadas referenciando al capítulo de planificación para más detalles: capítulo 6, página 17.

Viabilidad y planificación

X^{xxx,...}

6.1 Análisis de viabilidad

Aspectos a considerar

- Opcional pero recomendable.
- Su objetivo es realizar un análisis de la viabilidad del proyecto, validando sus posibles riesgos.
- Suele enfocarse descomponiéndolo en diferentes dimensiones: temporal (duración estimada para un TFG), tecnológica (relacionado con tecnologías usadas, normalmente opensource), económica (relacionado con tiempo de alumno en TFG, recursos requeridos que normalmente es HW ya disponible, el SW a realizar y otro SW normalmente opensource), ...

6.2 Planificación

Aspectos a considerar

- Enumeración de tareas en las diferentes iteraciones: Diagrama de Gantt
- Planificación inicial

6.3 Seguimiento

Aspectos a considerar

- Seguimiento de la planificación, y explicación de posibles retrasos

6.4 Análisis de costes

Aspectos a considerar

- Costes: Directos: personal, HW; Indirectos: luz, agua, ... ; gastos reserva contingencia; importante pro-ratear el precio de HW que no se amortiza en 6 meses, sino que es válido durante 4 o 5 años.
- Importante referenciar una fuente de la que se obtiene el coste por horas de determinado perfil

Requisitos del sistema

X^{xxx,...}

7.1 Introducción

Aspectos a considerar

- Transcripción textual/completa de los requisitos, punto de partida para identificar actores y a continuación casos de uso (siguientes secciones).
- Pueden diferenciarse entre requisitos funcionales y no funcionales

7.2 Actores

7.3 Casos de uso

Aspectos a considerar

- Agrupar casos de uso (servirá de base para agruparlos en servicios posteriormente en la sección de diseño).
- Definición concreta /detallada de cada caso de uso (precondiciones ...)

7.4 Modelo de casos de uso

Aspectos a considerar

- Diagrama de casos de uso incluyendo actores y relaciones entre casos

7.5 Prototipado de interfaz

Es interesante hacer el esfuerzo en este momento de prototipar las principales pantallas de la interfaz (y añadirlas en la memoria), como parte del análisis de requisitos, así como de aspectos de usabilidad de las mismas.

X^{xxx,...}

8.1 Introducción y objetivos

8.2 Resumen de patrones usados

8.3 Arquitectura general

Aspectos a considerar

- Arquitectura aplicación SPA
- División en subsistemas: En una aplicación SPA al menos backend y frontend pero podría haber más si se decide implementar utilidades e.g. como módulos independientes (e.g. como proyectos maven / jars independientes)

8.4 Subsistema Backend

8.4.1 Objetivos

8.4.2 Arquitectura

Aspectos a considerar

- Diagrama de paquetes general del subsistema (figura y descripción de la misma)

8.4.3 Modelo del dominio

Diagrama de entidades

Aspectos a considerar

- Diagrama de clases persistentes de la aplicación
- Para cada indicar, tanto de su semántica como de sus atributos.
- Justificar decisiones de diseño (¿desnormalizaciones?, métodos de lógica - Domain Model Pattern - ?)

Modelo de datos

Aspectos a considerar

- Diagrama entidad-relación
- Diccionario de Datos (en su ausencia - por temas de espacio bastante razonable - compensable con definición clara en diagrama de entidades).

Consideraciones adicionales de modelado de datos

Aspectos a considerar

- Claves primarias autogeneradas
- Anotaciones para relaciones entre entidades
- Optimistic Locking y @Version en Hibernate
- Relaciones LAZY/EAGER
- Optimizaciones hibernate en navegación entre entidades
- Optimizaciones hibernate - cache primer nivel

8.4.4 Capa de acceso a datos

Aspectos a considerar

- Arquitectura de un DAO
- Aspectos más relevantes de los DAOs implementados

8.4.5 Capa de lógica de negocio

Aspectos a considerar

- Arquitectura de servicios del modelo: Interfaces + clases asociadas
- Incluir descripción de métodos (en algunos casos podrían comentarse de forma agregada algunas operaciones por ejemplo gestión de usuarios (añadir/eliminar/modificar) en lugar de comentar de uno en uno.
- Incluir descripción de algoritmos en los casos que sea necesario. Podría incluirse un diagrama de secuencia para complementar algún algoritmo.

8.4.6 Capa servicios

Aspectos a considerar

- Diseño controladores
- API REST (intentar que sea REST-full cuando haya un mapping directo ... overloading del POST en otro caso
- Gestión de errores y aspectos de internacionalización de mensajes de error
- Autenticación y aspectos de control de acceso a los diferentes endpoints (SecurityConfig). Soporte de redirección tras autenticación, si implementado.

8.4.7 Otros?

Aspectos a resaltar

- Integración con otros sistemas o librerías a nivel de backend, e.g. pago con PayPal.
- Tb se podrían destacar aspectos como chats con websockets o algoritmos. Importante dejar claro todo lo que se comente en "objetivos" y "aportaciones" principales del proyecto.

8.5 Subsistema Frontend

8.5.1 Objetivos

8.5.2 Arquitectura

Aspectos a considerar

- Arquitectura de capas: al menos capa de acceso al servicio y capa de componentes de interfaz

8.5.3 Capa de acceso al servicio**8.5.4 Capa de componentes de interfaz****8.5.5 Otros?****Aspectos a considerar**

- Internacionalización de mensajes de texto
- Integración con otros sistemas / librerías

Implementación

X^{xxx,...}

9.1 Software requerido

Pre-requisitos software para poder compilar el código fuente. Respecto a la base de datos, puede referenciarse al apéndice de instrucciones de instalación, a la parte de instalación y configuración de la base de datos.

9.2 Estructura

Estructura de carpetas del código fuente. Si se utiliza maven, básicamente indicar estructura estándar de maven + casos especiales como localización de fichero de creación de datos ...

9.3 Instrucciones de compilación

Instrucciones de cómo compilar el código fuente. Si se usa maven, básicamente comentar las fases principales a usar de maven.

Comentar también cómo ejecutar en desarrollo.

Capítulo 10

Pruebas

X^{xxx,...}

10.1 Introducción

Aspectos a considerar

- Importancia de las pruebas, tipos de pruebas realizadas en este proyecto y metodología de las pruebas.

10.2 Pruebas unitarias

10.3 Pruebas de integración

Aspectos a considerar

- Importante indicar si son automatizadas o manuales.
- En el caso de pruebas del modelo, casi seguro que serán automatizadas. Necesario indicar la idea de las pruebas, repetibles para detectar posibles problemas de regresión en el futuro e independientes unas de otras: cada prueba crea lo necesario para validar el caso de uso, invoca el caso de uso a probar y valida los efectos asociados a la invocación del caso de uso y eliminar todos los datos generados. Importante validar diferentes escenarios de éxito y fallo posibles para cada caso de uso.
- En las pruebas de integración, Recomendable incluir el nivel de cobertura de las mismas(existen plugins que lo calculan de forma sencilla).
- En el caso de pruebas de capa web, si no se han automatizado será necesario describir el plan de pruebas.

10.4 ...

Conclusiones y futuras líneas de trabajo

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras, (no caso de TFM, carácter profesional do mesmo)...

11.1 Conclusiones

11.2 Aprendizajes

11.3 Futuras líneas de trabajo

Apéndices

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

A.1 Instalación del Software

A.2 Manual de usuario

Introducción al layout general de la aplicación, y subapartados para cada una de las secciones principales, para que sea más sencillo de consultar / comprender.

Lista de acrónimos

API Application Programming Interface. [7](#), [8](#), [11](#)

AXEGA Axencia Galega de Emerxencias (112 Galicia). [1](#), [2](#)

Compose Jetpack Compose. [9](#)

CSS Cascading Style Sheet. [5](#), [6](#), [13](#)

EL Java Expression Language. [12](#)

FCM Firebase Cloud Messaging. [3](#), [10](#)

FMS FirebaseMessagingService. [10](#)

HTML HyperText Markup Language. [5](#), [6](#), [13](#)

HTTP Hypertext Transfer Protocol. [10](#), [11](#)

[. 9](#)

HTTPS Hypertext Transfer Protocol Secure. [10](#)

IDE Integrated Development Environment. [11](#)

IoC Inversión de Control. [7](#)

JDBC Java Database Connectivity. [7](#)

JPA Java Persistence API. [7](#)

JSON JavaScript Object Notation. [6](#)

[. 3](#)

JSP JavaServer Pages. [12](#)

Material Material Design. [9](#)

PLADIGA Plan de Prevención y Defensa Contra los Incendios Forestales de Galicia. [2](#)

REST Representational State Transfer. [10](#)

SQL Structured Query Language. [5](#), [7](#)

TFG Trabajo Fin de Grado. [1-3](#)

TFM Trabajo Fin de Máster. [1](#), [2](#)

Bibliografía

- [1] M. para la Transición Ecológica y el Reto Demográfico (MITECO), “Estadística general de incendios forestales (egif),” 2025, consultado el 2026-04-29. [En línea]. Disponible en: <https://www.miteco.gob.es/es/biodiversidad/temas/incendios-forestales/estadisticas-datos.html>
- [2] A. G. de Emerxencias (AXEGA), “O 112 galicia xestionou unha decena de incidencias relacionadas coa chuvia en ourense, a rúa e o carballiño,” 2026, consultado el 2026-04-29. [En línea]. Disponible en: <https://www.axega112.gal/gl/content/o-112-galicia-xestionou-unha-decena-de-incidencias-relacionadas-coa-chuvia-en-ourense-rua-e>
- [3] —, “Alerta laranxa por chuvias intensas no noroeste de ourense,” 2026, consultado el 2026-04-29. [En línea]. Disponible en: https://www.axega112.gal/gl/alertas_activas
- [4] —, “Rexistrado un sismo de magnitude 2.6 con epicentro en ponteceso,” 2026, consultado el 2026-04-29. [En línea]. Disponible en: <https://www.axega112.gal/gl/content/rexistrado-un-sismo-de-magnitude-26-con-epicentro-en-ponteceso>
- [5] A. García Vázquez, “Aplicación para coordinación de equipos para la prevención y extinción de incendios forestales,” Memoria de TFG, Universidade da Coruña, 2023. [En línea]. Disponible en: <https://ruc.udc.es/entities/publication/04b8901f-0960-4d07-8cce-9a2ecfcfb471>
- [6] A. G. de Emerxencias (AXEGA), “Portal oficial de axega e 112 galicia,” 2026, consultado el 2026-04-29. [En línea]. Disponible en: <https://www.axega112.gal>
- [7] X. e. D. X. d. G. Consellería de Presidencia, “Plans de emerxencia de galicia,” 2026, consultado el 2026-04-29. [En línea]. Disponible en: <https://cpxt.xunta.gal/plans-de-emerxencia>
- [8] C. do Medio Rural e Desenvolvemento Sostible. Xunta de Galicia, “Pladiga: Plan de prevención e defensa contra os incendios forestais (pladiga),” 2026, consultado el 2026-

- 04-29. [En línea]. Disponible en: <https://mediorural.xunta.gal/es/temas/defensa-monte/pladiga-2026>
- [9] J. del Estado, “Ley 17/2015, de 9 de julio, del sistema nacional de protección civil,” 2015, consultado el 2026-04-29. [En línea]. Disponible en: <https://www.boe.es/eli/es/l/2015/07/09/17/con>
- [10] M. del Interior, “Real decreto 524/2023, de 20 de junio, por el que se aprueba la norma básica de protección civil,” 2023, consultado el 2026-04-29. [En línea]. Disponible en: <https://www.boe.es/eli/es/rd/2023/06/20/524/con>
- [11] Oracle, “Java,” <https://www.oracle.com/java/>, consultado el 2026-04-29.
- [12] JetBrains, “Kotlin programming language,” <https://kotlinlang.org/>, consultado el 2026-04-29.
- [13] W3Schools, “Sql,” <https://www.w3schools.com/sql/>, consultado el 2026-04-29.
- [14] M. W. Docs, “Html y css,” https://developer.mozilla.org/es/docs/Learn/Getting_started_with_the_web/HTML_basics, consultado el 2026-04-29.
- [15] Mozilla, “Javascript,” <https://developer.mozilla.org/es/docs/Web/JavaScript>, consultado el 2026-04-29.
- [16] “Jsx (javascript xml),” <https://reactjs.org/docs/introducing-jsx.html>, accedido el 2026-04-29.
- [17] “Latex,” <https://www.latex-project.org/help/documentation/>, accedido el 2026-04-29.
- [18] “Json (javascript object notation),” <https://www.json.org/>, accedido el 2026-04-29.
- [19] Spring, “Spring,” <https://spring.io/>, accedido el 2026-04-29.
- [20] —, “Spring boot,” <https://spring.io/projects/spring-boot>, consultado el 2026-04-29.
- [21] —, “Spring data,” <https://spring.io/projects/spring-data>, consultado el 2026-04-29.
- [22] Oracle, “Java persistence api,” <https://docs.oracle.com/javaee/6/tutorial/doc/bnbpz.html>, consultado el 2026-04-29.
- [23] Hibernate, “Hibernate orm,” <https://hibernate.org/orm/>, consultado el 2026-04-29.
- [24] JDBC, “Jdbc,” <https://www.ibm.com/docs/es/developer-for-zos/9.5.1?topic=support-what-is-jdbc>, consultado el 2026-04-29.
- [25] Facebook, “React,” <https://reactjs.org/>, consultado el 2026-04-29.

- [26] Redux, “Redux,” <https://redux.js.org/>, consultado el 2026-04-29.
- [27] R. Toolkit, “Redux toolkit query,” <https://redux-toolkit.js.org/rtk-query/overview>, consultado el 2026-04-29.
- [28] Material-UI, “Material-ui,” <https://material-ui.com/>, consultado el 2026-04-29.
- [29] R. M. GL, “React map gl,” <https://visgl.github.io/react-map-gl/>, consultado el 2026-04-29.
- [30] O. W. Map, “Open weather map,” <https://openweathermap.org/>, consultado el 2026-04-29.
- [31] Mapbox, “Mapbox,” <https://www.mapbox.com/>, accedido el 2026-04-29.
- [32] G. Inc., “Gradle build tool,” <https://gradle.org/>, consultado el 2026-04-29.
- [33] A. Developers, “Jetpack compose,” <https://developer.android.com/jetpack/compose>, consultado el 2026-04-29.
- [34] Google, “Material design,” <https://material.io/>, consultado el 2026-04-29.
- [35] P. EmergencyApp, “HttpClient (proyecto),” <https://github.com/your/repo>, consultado el 2026-04-29.
- [36] G. Firebase, “Firebase cloud messaging (fcm),” <https://firebase.google.com/docs/cloud-messaging>, consultado el 2026-04-29.
- [37] A. D. . Firebase, “Firebasemessaging (android),” <https://firebase.google.com/docs/cloud-messaging/android/receive>, consultado el 2026-04-29.
- [38] JUnit.org, “Junit,” <https://junit.org/junit5/>, consultado el 2026-04-29.
- [39] Mozilla, “Http,” <https://developer.mozilla.org/es/docs/Web/HTTP>, consultado el 2026-04-29.
- [40] REST, “Rest,” <https://restfulapi.net/>, accedido el 2026-04-29.
- [41] JetBrains, “Intellij,” <https://www.jetbrains.com/idea/>, consultado el 2026-04-29.
- [42] Postman, “Postman,” <https://www.postman.com/>, consultado el 2026-04-29.
- [43] Microsoft, “Vscode,” <https://code.visualstudio.com/>, consultado el 2026-04-29.
- [44] DBeaver, “Dbeaver,” <https://dbeaver.io/>, consultado el 2026-04-29.
- [45] Git, “Git,” <https://git-scm.com/>, consultado el 2026-04-29.

- [46] “Apache maven,” <https://maven.apache.org/>, Apache Software Foundation, accedido el 2026-04-29.
- [47] Facebook, “Create react app,” <https://create-react-app.dev/>, consultado el 2026-04-29.
- [48] P. G. D. Group, “Postgresql,” <https://www.postgresql.org/>, consultado el 2026-04-29.
- [49] P. D. Team, “Postgis,” <https://postgis.net/>, consultado el 2026-04-29.
- [50] T. A. S. Foundation, “Apache tomcat,” <https://tomcat.apache.org/>, consultado el 2026-04-29.
- [51] O. S. Community, “Webpack,” <https://webpack.js.org/>, consultado el 2026-04-29.
- [52] Serve, “Serve,” <https://www.npmjs.com/package/serve?activeTab=readme>, consultado el 2026-04-29.
- [53] I. Docker, “Docker,” <https://www.docker.com/>, consultado el 2026-04-29.